Exercise 1 – Basics of containers and microservices **Tripti Bara, K428980**

Platform Information

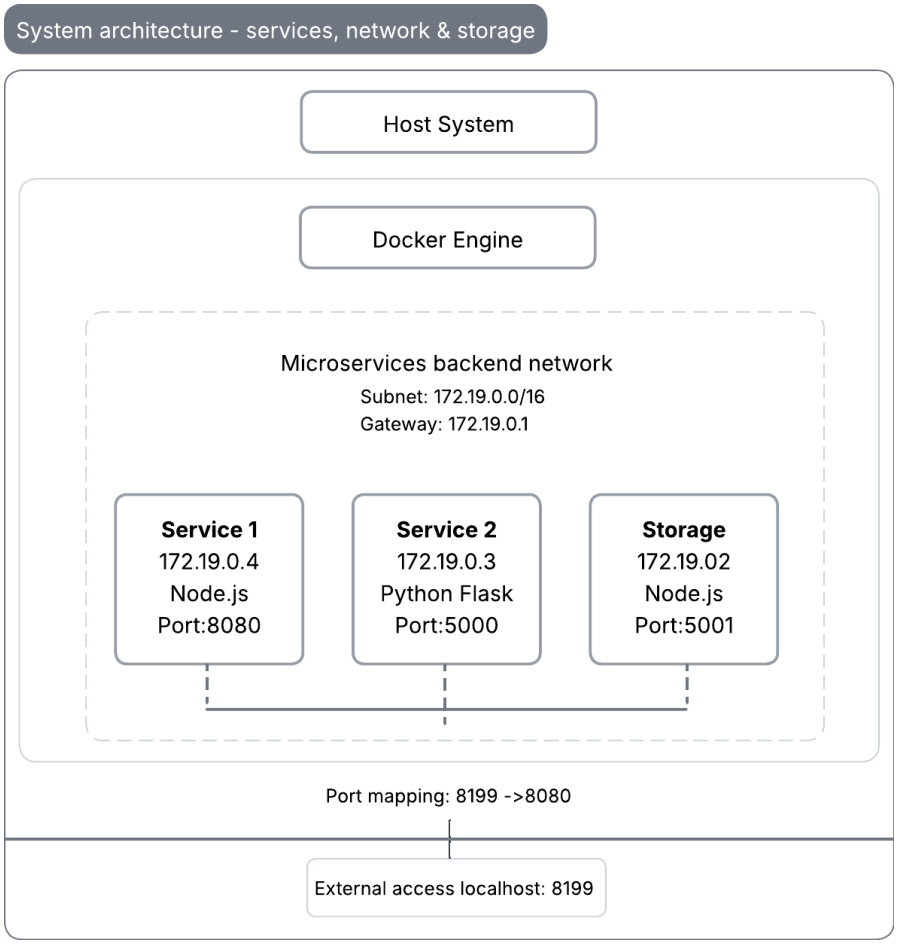
Hardware: Physical machine

Operating System: Microsoft Windows 11 Pro (x64-based PC)

Docker Version: 27.5.1, build 9f9e405

Docker Compose version: v2.32.4-desktop.1

System Architecture: services, network & storage



Service Communication Flow:

1. Client - service1:8080 (via localhost:8199)

2. service1 - storage:5001 (log storage)

3. service1 - service2:5000 (status request)

4. service2 - storage:5001 (log storage)

Volume Mounts:

- vStorage: /vstorage (shared between service1 and service2)

- storage_data: /data (persistent storage for storage service)

Analysis of Status Records

Timestamp1: 2025-09-24T15:12:33Z: uptime 1.50 hours, free disk in root: 968976 MBytes

Timestamp2: 2025-09-24T15:12:33Z: uptime 1.5 hours, free disk in root: 968976 Mbytes

What is Actually Measured?

Disk Space Measurement

- **Measured Value:** ~968,976 MB (≈946 GB) free disk space

- **Measurement Location:** Container's root filesystem (`/`)

- **Method:** Unix `df` command with `-BM` flag

- **Relevance: Low relevance** - This measures the Docker container's overlay filesystem, not the host system's actual disk usage

Uptime Measurement

- **Measured Value:** ~1.5 hours at time of testing

- **Measurement Source:** Container's `/proc/uptime` file

- **Relevance: Low relevance** - Measures container uptime, not host system uptime or actual service availability

Current Issues:

1. Measuring container filesystem rather than meaningful application metrics

2. Only basic system metrics, no application-specific metrics

3. No validation of metric collection failures

Recommended Improvements:

1. Monitor request count, response times, error rates

2. Use Docker API or host agents to gather meaningful resource usage

3. Monitor storage service performance, request queue depth

4. Use JSON format for better parsing and analysis

Persistent Storage Solutions Analysis

Solution 1: Docker Named Volumes (`vStorage` and `storage_data`)

Advantages

- Docker-managed: Handles lifecycle automatically
- Performance: Uses optimized Docker storage
- Portable: Runs on any host OS
- Backup-friendly: Easy docker volume backups
- Shared: Many containers can use one volume

Disadvantages

- Hidden from host: Not directly accessible
- Less control: Limited location and permission options
- Migration needed: Requires Docker commands
- Hard to debug: Data inspection needs container access

Solution 2: Direct File System Writes (`/vstorage/log.txt`)

Advantages

- Simple: Direct file access
- Fast: No extra layers
- Debuggable: Easy to inspect inside containers

Disadvantages

- No persistence: Data lost with container removal
- Isolated: Not shared between containers
- Limited: Restricted by container storage
- Unscalable: Cannot support multiple instances

Solution 3: REST API Storage (Storage Service)

Advantages

- Centralized: One source for all logs
- Scalable: Supports many clients
- API-based: Uniform data access
- Resilient: Handles network errors

Disadvantages

- Network-bound: Needs connectivity
- Failure risk: One service can halt system
- Latency: Extra network overhead
- Complex: More to maintain

Recommendation:

Hybrid approach: combining Docker volumes for persistence with REST API for coordination provides the best balance of reliability, performance, and maintainability.

Cleaning Up Persistent Storage

- Stop and remove all containers: docker-compose down
- Remove all volumes: docker volume rm microservices_vStorage microservices_storage_data
- Remove all images: docker-compose down --rmi all
- Remove unused networks: docker network prune -f

Difficulties Encountered

- Using different programming languages for different services was a little challenging but a good learning experience
- Code changes were not reflecting in running containers due to Docker image caching and needed rebuilding. Using ‘docker-compose up—build’ helped in image rebuild.
- Understanding the system initially was challenging.